Finite State Machines

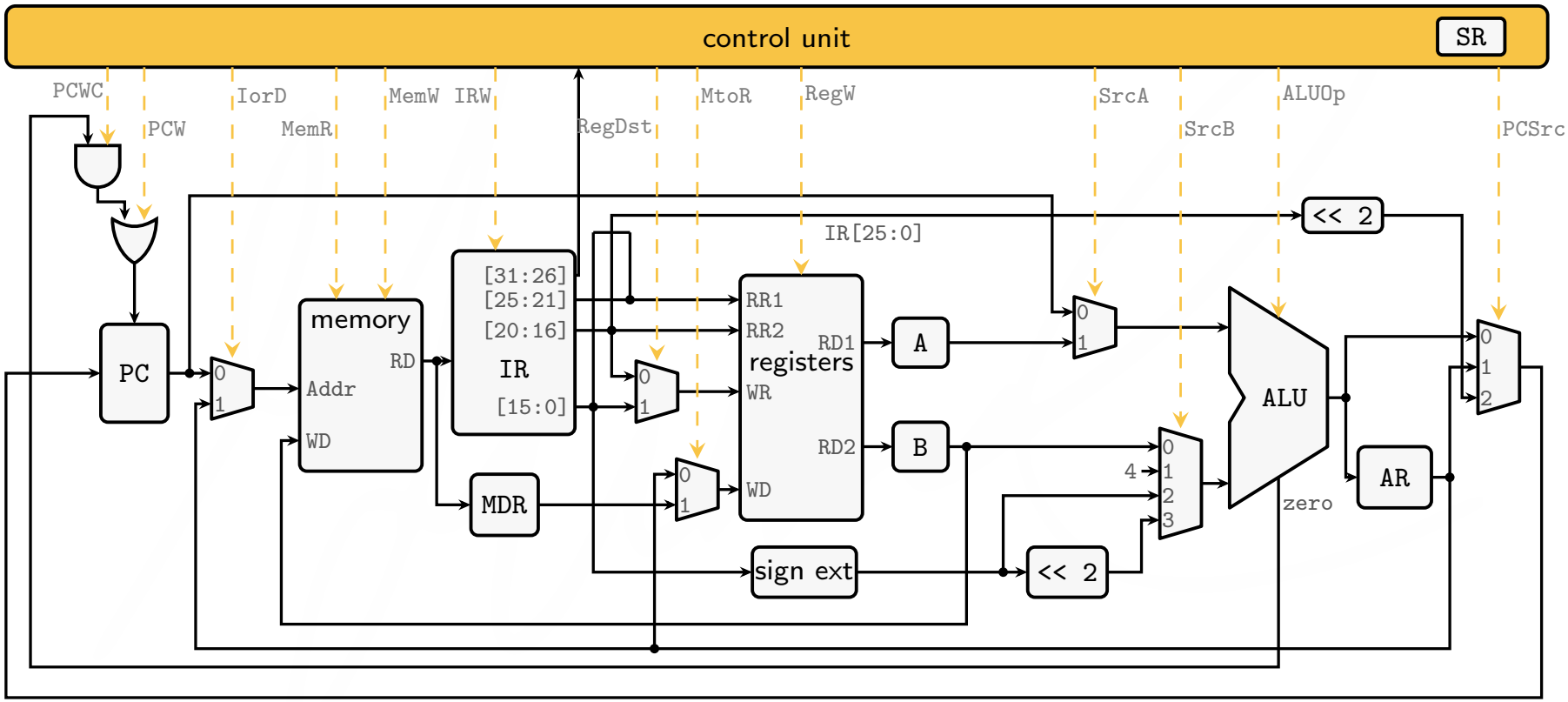
Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io



The last part of the computer is the **control unit**. Every clock cycle, it sets the **control signals** (dashed) that tell each part what to do.



To build it, we need a **finite state machine**.

finite state machine

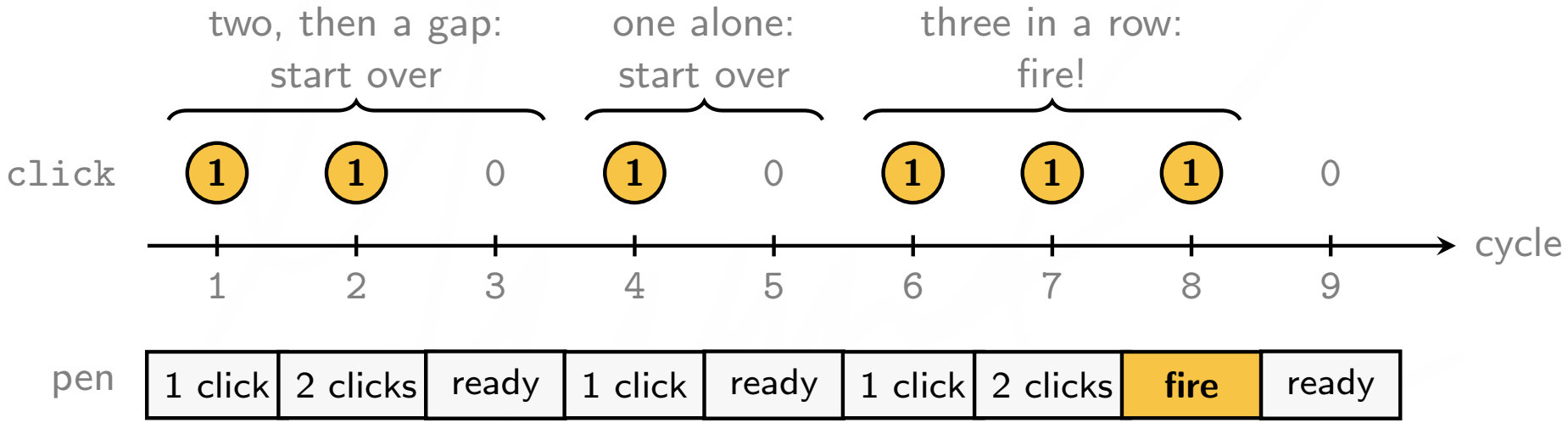
a circuit that moves through a fixed set of states as inputs arrive

A finite state machine is described by three things:

1. **states**
   the situations the machine can be in
2. **transitions**
   how each input moves it between states
3. **outputs**
   what it produces in each state

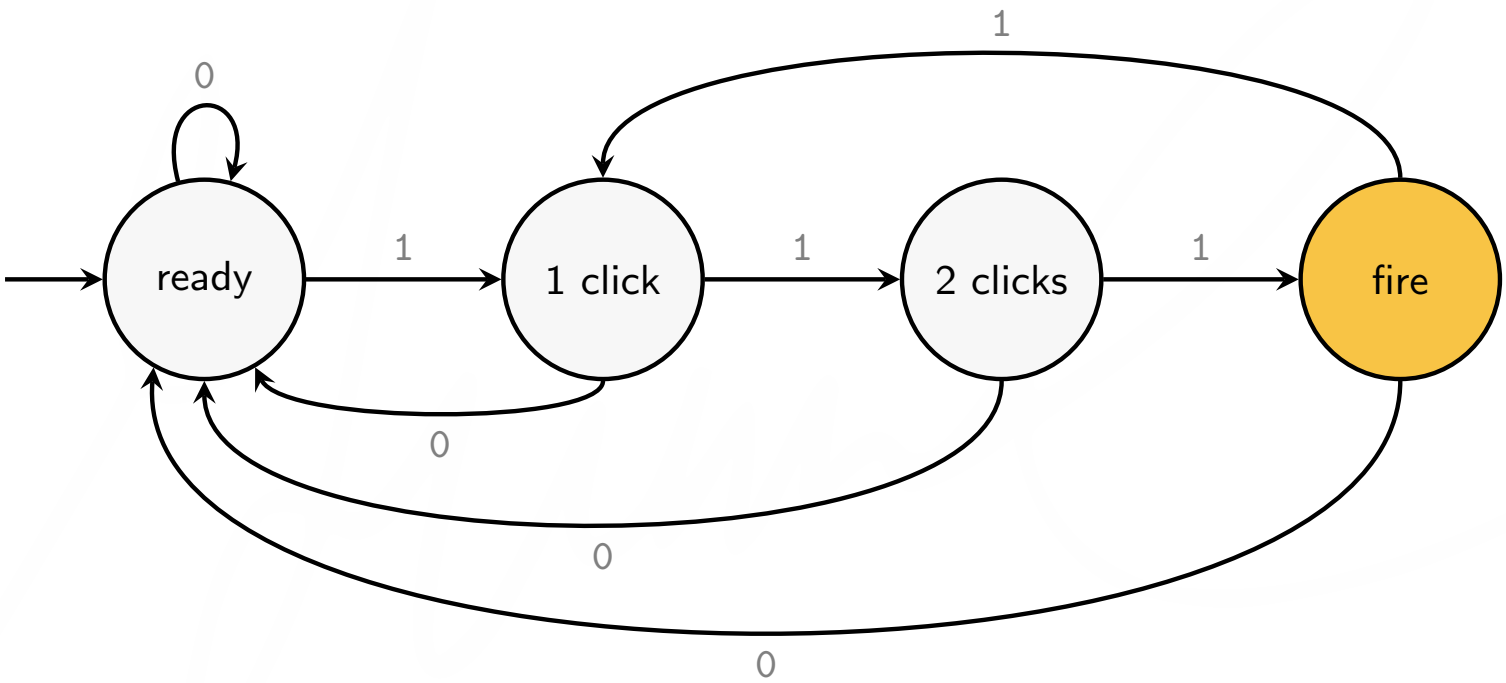
state diagrams

Our running example is a spy's **pen** that fires a dart. It has one button, and on every clock cycle it reads `click`: `1` if the button was clicked, `0` if not. Three clicks in a row fire the dart.



The pen's output, `fire`, is `1` in the cycle the dart fires.

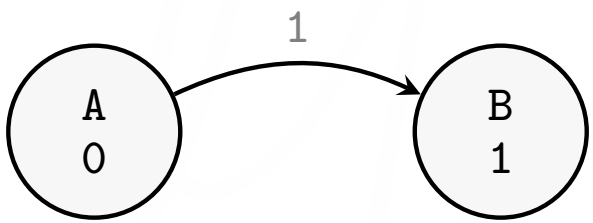
A **state diagram** draws each state as a circle and each transition as an arrow labelled with the input that causes it.



There are two styles of machine, differing in where the output comes from.

Moore machine

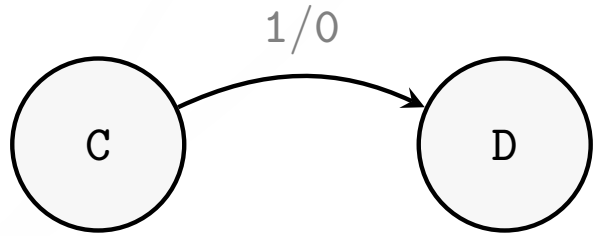
the output depends only on the current state



output written
inside the state

Mealy machine

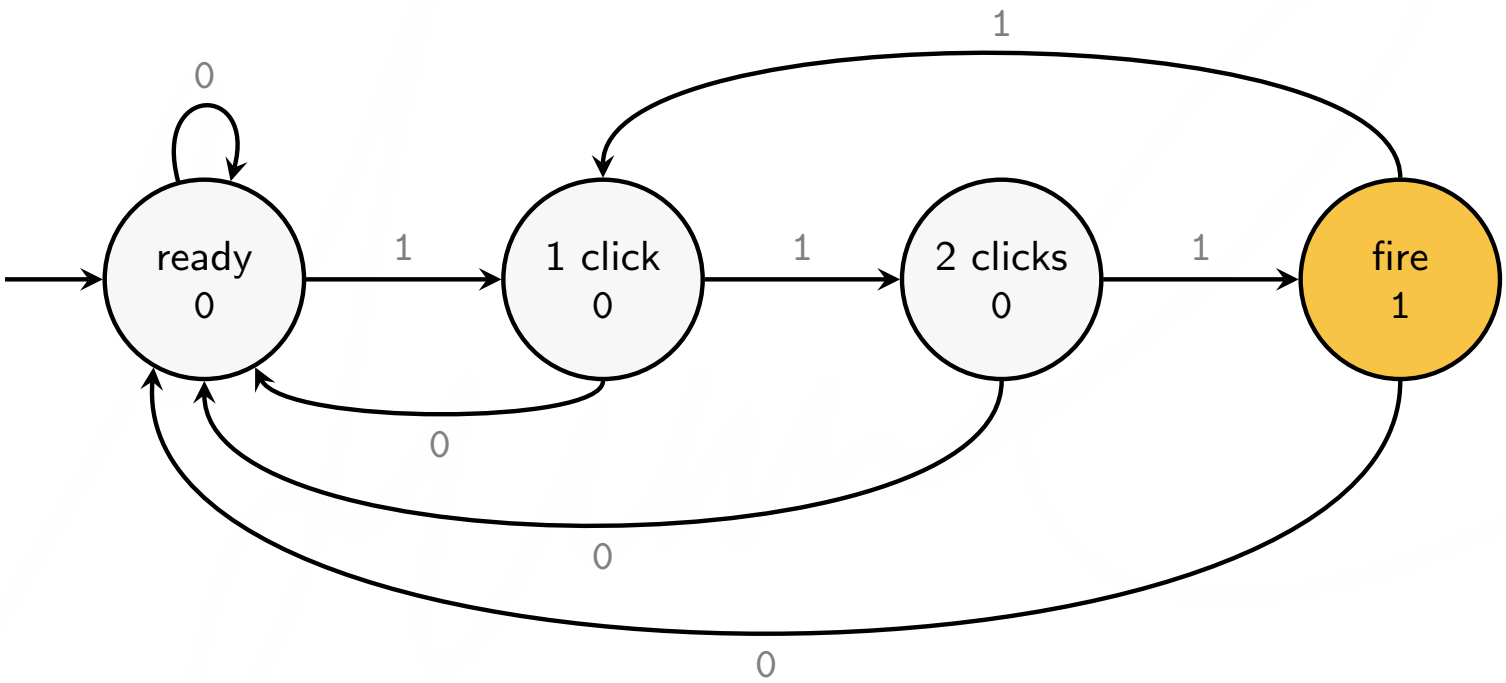
the output depends on the state and the input



output written
on the arrow (input/output)

We use Moore machines, so each state carries its own output.

In a Moore machine, we write each state's output inside its circle. The pen's output, `fire`, is 1 only in the fire state.



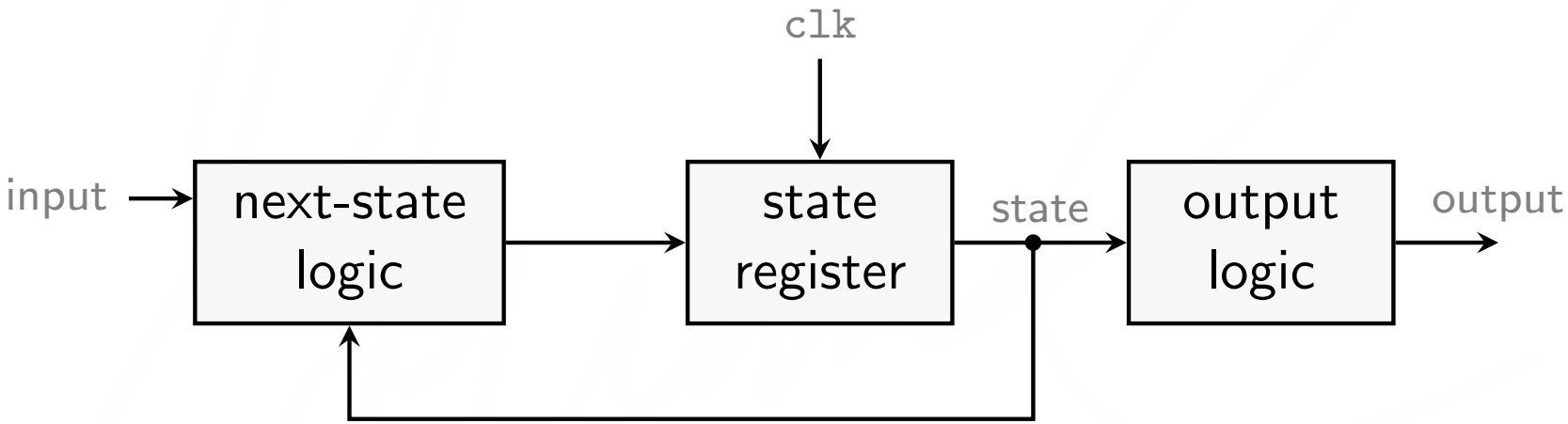
Clicking 1, 1, 1, 1, 0, 1, 1, 1 moves the pen through these states:

cycle	`click`	state	`fire`
start		ready	0
1	1	1 click	0
2	1	2 clicks	0
3	1	fire	1
4	1	1 click	0
5	0	ready	0
6	1	1 click	0
7	1	2 clicks	0
8	1	fire	1

After firing, the next click starts a new count; a 0 always sends the pen back to ready.

structure

Every finite state machine has the same shape: a register holds the state, and combinational logic computes the next state and the output.



The number of flip-flops follows from the number of states, since each flip-flop stores one bit.

$$\#\text{ of flip-flops} = \lceil \log_2(\text{states}) \rceil$$

the design procedure

Designing any finite state machine follows the same steps.

(1) **state diagram**

draw the state diagram

(2) **state table**

write the state table

(3) **bit pattern**

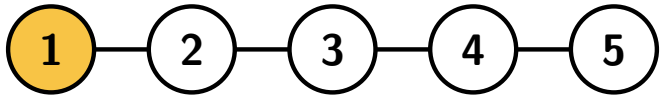
assign a bit pattern to each state

(4) **truth table**

rewrite the table with those patterns

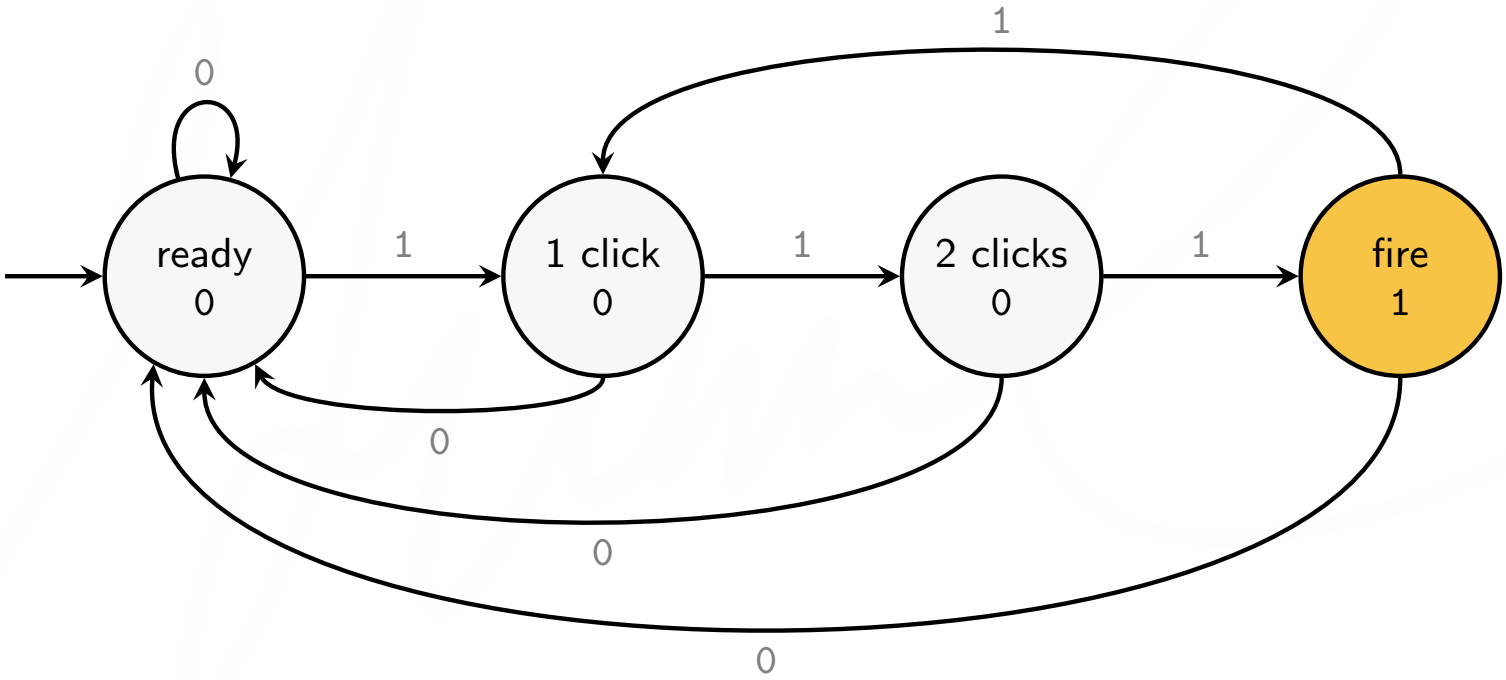
(5) **circuit design**

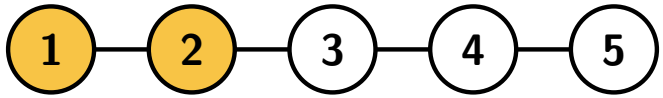
design the logic for the outputs and each flip-flop input



draw the state diagram

We already have it: four states, with the output inside each one.





write the state table

Each arrow in the diagram becomes one row: where the pen goes from each state, for each input.

state	`click`	next state	`fire`
ready	0	ready	0
ready	1	1 click	0
1 click	0	ready	0
1 click	1	2 clicks	0
2 clicks	0	ready	0
2 clicks	1	fire	0
fire	0	ready	1
fire	1	1 click	1



assign a bit pattern to each state

We give each state a bit pattern. Counting in binary seems natural:

state	Q_1	Q_0
ready	0	0
1 click	0	1
2 clicks	1	0
fire	1	1

Which pattern we assign to each state matters. Going from 1 click (01) to 2 clicks (10) changes two bits. They may not change at the same instant, so the pen can pass through a wrong state on the way.



assign a bit pattern to each state

Instead, we assign patterns so each click flips a single bit:

state	Q_1	Q_0
ready	0	0
1 click	0	1
2 clicks	1	1
fire	1	0

Every state can go back to ready, so two transitions still flip both bits: 2 clicks to ready, and fire to 1 click. With two flip-flops, no assignment avoids that.



rewrite the table with those patterns

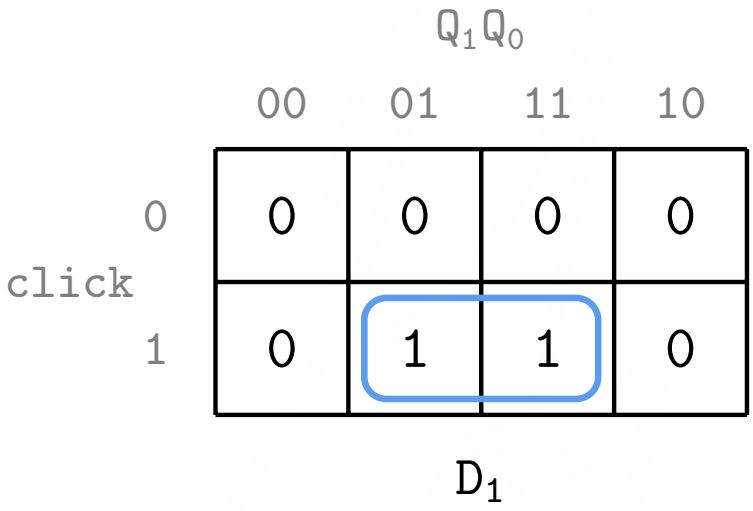
Replacing each state by its pattern gives a truth table. The next state is what we load into the flip-flops, so it gives D_1 and D_0.

	Q_1	Q_0	click	D_1	D_0	fire
ready	0	0	0	0	0	0
ready	0	0	1	0	1	0
1 click	0	1	0	0	0	0
1 click	0	1	1	1	1	0
2 clicks	1	1	0	0	0	0
2 clicks	1	1	1	1	0	0
fire	1	0	0	0	0	1
fire	1	0	1	0	1	1

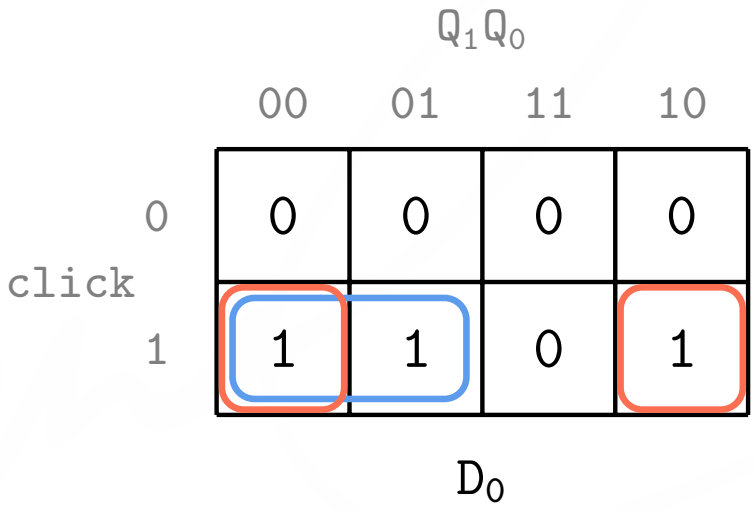


design the logic

A Karnaugh map for each flip-flop input finds its logic.



$Q_0 \cdot \texttt{click}$



$\overline{Q_1} \cdot \texttt{click} + \overline{Q_0} \cdot \texttt{click}$

The dart fires exactly in state 10, so $\texttt{fire} = Q_1 \cdot \overline{Q_0}$.



design the logic

Together, the maps give the logic for each part of the machine:

next-state logic

$$D_1 = Q_0 \cdot \texttt{click}$$

an AND gate

$$D_0 = \overline{Q_1} \cdot \texttt{click} + \overline{Q_0} \cdot \texttt{click}$$

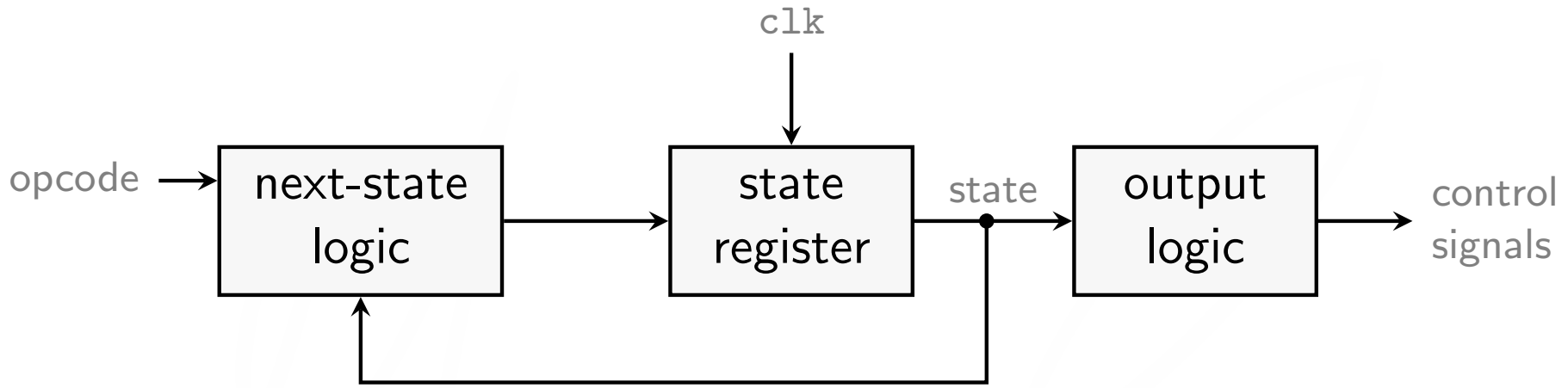
1 on a click, except from 2 clicks (11)

output logic

$$\texttt{fire} = Q_1 \cdot \overline{Q_0}$$

one AND gate, with Q_0 inverted

The control unit is a finite state machine with the same shape.



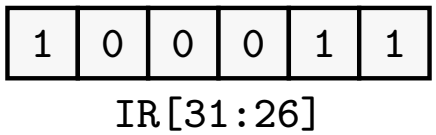
state

which step of the current instruction we are on



inputs

the opcode from the IR



outputs

the control signals, one set per state



A finite state machine combines everything so far: gates compute the next state, and a register remembers it. In a processor, this is the control unit.